Graded Project – DevAnswers (React 2)

Introduction

In this project, you will complete the state management layer of **DevAnswers** - a full-stack Q&A web application for developers. The frontend is built with **React.js** and **Redux Toolkit** and connects to a real Express/MongoDB backend via service functions. Your task is to implement both the **Redux state management logic** and several other UI rendering tasks described in this document.

This project will help you practice:

- Managing global application state with **Redux Toolkit**
- Writing **async thunks** (`createAsyncThunk`) to call API services
- Handling loading and error states with `extraReducers` / `builder`
- Persisting auth state to `localStorage`
- Dispatching actions from React components
- Rendering dynamic lists with `.map()` and conditional display
- Using Github Copilot to help with all the tasks above

Setup and Execution

1. Download the project zip file and extract it. It will have both the frontend and backend code in separate folders. You will not need to make any changes to the backend for this project (except for setting up the .env file with appropriate environment variables). Run the backend server first by setting up the .env file with appropriate environment variables and then running `npm install`, `npm run populate`, and finally `npm run dev`.

2. Navigate to the frontend folder:

   ```
   cd devanswers-frontend
   ```

3. Install dependencies:

   ```
   npm install
   ```

4. Start the development server:

   ```
   npm run dev
   ```

5. Open the app in your browser at `http://localhost:5173`. At this point the app is not functional. You will see a blank page. The DevTools console will show errors related to the unimplemented functions. Your task is to implement these missing pieces as described in the next sections.

Project Structure - Files to Implement

You will have to implement the following in the project:

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

```
devanswers-frontend/
├── src/
│   ├── components/
│   │   └── Header/
│   │       └── Header.jsx
│   ├── pages/
│   │   └── Auth/
│   │       └── Login.jsx
│   ├── reducers/
│   │   └── questionSlice.js
│   └── services/
│       ├── authService.js
│       └── questionService.js
└── tests/
    ├── unit/
    │   └── reducers/
    │       └── questionSlice.test.jsx
    └── integration/
        └── questionFlow.test.jsx
```

Problem Statement

Part 1: Service Layer (`src/services/`)

Review the API routes in the file src/config/config.js to understand the backend endpoints. Use the `axiosInstance` from `src/api/axiosInstance.js` to make API calls in the service functions.

a. **authService.js**: This file handles authentication API calls. Implement the following functions to call the backend API. For each function, return `res.data.data` from the API response.

- `login(credentials)` — to POST to the login endpoint;
- `register(formData)` — to POST to the register endpoint;

b. **questionService.js**: This file handles question-related API calls. Implement the following functions to call the backend API. For each function, return `res.data.data` from the API response.

- `getAllQuestions()` — to GET all questions;
- `getQuestionById(id)` — to GET a single question by ID;
- `upvoteQuestion(questionId, token)` — to POST to the upvote endpoint with an empty body and `Authorization` header;
- `downvoteQuestion(questionId, token)` — to POST to the downvote endpoint with an empty body and `Authorization` header;
- `createQuestion(questionData, token)` — to POST the new question with `Authorization` header;
- `getAnswersByQuestionId(questionId)` — to GET answers for a question;

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

- `createAnswerForQuestion(questionId, answerText, token)` — to POST a new answer to a question with `Authorization` header;

After implementing the above service functions, you will still not see any changes in the UI since we haven't connected these services to our Redux state or React components yet. The next steps will involve implementing the Redux slices and connecting them to the components.

Part 2: Components (`src/components/Header/`, `src/pages/Auth/`)

a. **Header.jsx** (`src/components/Header/Header.jsx`): This component renders the application header with logout functionality.

- Inside `handleLogout`, dispatch the `logoutUser` thunk before calling `navigate('/')`. The `logoutUser` method clears `localStorage` and resets Redux state.

b. **Login.jsx** (`src/pages/Auth/Login.jsx`): This page handles user login via Redux.

- Dispatch `loginUser({ email, password })` inside `handleSubmit`. The `loginUser` thunk calls the login API and saves the user info to Redux and `localStorage` on success.

After implementing the above, you should be able to log in and see the header update with the user's name and a logout button. Logging out should clear the user info and navigate back to the login page.

Part 3: `questionSlice` (`src/reducers/questionSlice.js`)

Implement the following thunks using `createAsyncThunk`. For all the thunks, use `rejectWithValue` on failure.

a. `postQuestion` thunk

- Get `userInfo` from `getState().user.userInfo`
- Call `createQuestion` with { title, description, tags, author: userInfo.userId } as the question data. Also pass in the token from userInfo.token`

b. `voteQuestion` thunk

- Get `token` from `getState().user.userInfo`
- Call `upvoteQuestion(question._id, token)` or `downvoteQuestion(question._id, token)` depending on `voteType`

c. `voteAnswer` thunk

- Get `token` from `getState().user.userInfo`
- Call `upvoteAnswer(answer._id, token)` or `downvoteAnswer(answer._id, token)` depending on `voteType`

d. `postAnswer` thunk

- Get `userInfo` from `getState().user.userInfo`

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

- Call `createAnswerForQuestion` with appropriate parameters.

Also implement **`extraReducers`** for votes and answers. Add `pending`, `fulfilled`, and `rejected` cases for `voteQuestion`, `voteAnswer`, and `postAnswer`:

- `voteQuestion.fulfilled` → update the upvotes, downvotes and votecount of the matching question in `state.questions` and `state.currentQuestion`
- `voteAnswer.fulfilled` → update the upvotes, downvotes and votecount of the matching answer inside `state.currentQuestion.answers` by first finding the index of the answer from the answer ID.
- `postAnswer.fulfilled` → push the new answer into `state.currentQuestion.answers`

After implementing the above thunks and reducers, you should be able to post new questions and answers, and upvote/downvote existing ones. The UI should update in real-time to reflect the changes in votes and new answers.

Part 4: Tests

- **`tests/unit/reducers/questionSlice.test.jsx`** — Create unit tests for the `questionSlice` reducer. Tests initial state and `pending` / `fulfilled` / `rejected` transitions for `fetchQuestions` and `postQuestion`. No API calls - directly invokes the reducer with action creators.
- **`tests/integration/questionFlow.test.jsx`** — Build integration tests that dispatch real Redux thunks against an MSW-intercepted API. Covers `fetchQuestions`, `fetchQuestionById`, `postQuestion`, and `voteQuestion`, and assert the resulting Redux state.

Run tests from the project root with the following command:

```
npm test
```

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.